Instituto Superior de Engenharia de Lisboa

Departamento de Engenharia Eletrónica e Telecomunicações e de Computadores (DEETC)

Mestrado em Engenharia Informática e de Computadores

PRACTICE 6 - Report

Train-Test Split Methods and Evaluation Metrics

Aprendizagem e Mineração de Dados (AMD)

1º Semestre – 2025/2026

Autores:

Florin Sofinet (A46608)
Diogo Guerra (A49149)
Nuno Silva (A53976)

December 1, 2025

Contents

1 Introduction **2**
1.1 Objectives . . . 2

2 Exercise 6: Building a Classifier with Train-Test Split **2**
2.1 Overview . . . 2
2.2 Implementation Steps . . . 2
2.2.1 Step a) Comment All Train-Test Split Methods . . . 2
2.2.2 Step b) Apply DecisionTreeClassifier . . . 3
2.2.3 Step c) Apply Holdout Split Method . . . 3
2.3 Holdout Split Method . . . 3
2.4 Results . . . 4
2.4.1 Analysis . . . 4

3 Exercise 7: Multiple Metrics and Repeated Splits **4**
3.1 Overview . . . 4
3.2 Implementation Steps . . . 4
3.2.1 Step d) Complete the score_recipe Function . . . 4
3.2.2 Step e) Test with Repeated Holdout . . . 5
3.3 Repeated Holdout Method . . . 5
3.4 Evaluation Metrics . . . 6
3.4.1 Accuracy . . . 6
3.4.2 Precision . . . 6
3.4.3 Recall (Sensitivity) . . . 6
3.4.4 F1-Score . . . 6
3.4.5 Cohen's Kappa . . . 6
3.5 Results with Simple Dataset . . . 7
3.5.1 Analysis of Results . . . 7
3.6 Understanding Metric Warnings . . . 8
3.6.1 Step f) & g) Warning Analysis . . . 8
3.6.2 Step h) Handling Warnings with zero_division Parameter . . . 9
3.6.3 Step i) Suppressing Warnings . . . 9

4 Comparative Analysis **9**
4.1 Impact of Dataset Size . . . 9
4.2 Holdout vs Repeated Holdout . . . 10

5 Conclusions **10**
5.1 Key Learnings . . . 10
5.2 Best Practices . . . 10
5.3 Future Work . . . 11

1 Introduction

This report documents the implementation and analysis of exercises 6 and 7 from Practice 6, focusing on building classifiers using various train-test split methods and evaluating their performance using multiple scoring metrics. The exercises utilize the `my_split_and_eval.py` script, which provides a comprehensive framework for machine learning model evaluation using scikit-learn.

1.1 Objectives

The main objectives of these exercises are:

- Implement and understand different train-test split methodologies
- Build classification models using Decision Tree Classifier
- Apply multiple evaluation metrics to assess model performance
- Understand and handle metric warnings in edge cases
- Analyze the impact of small datasets on evaluation metrics

2 Exercise 6: Building a Classifier with Train-Test Split

2.1 Overview

Exercise 6 focuses on the fundamental implementation of a classifier using a single train-test split method. The goal is to establish the basic pipeline for training and evaluating machine learning models.

2.2 Implementation Steps

2.2.1 Step a) Comment All Train-Test Split Methods

The first step required commenting out all train-test split methods in the `list_func_tt_split` list. This ensures a clean starting point where no split method is active.

```
# list_func_tt_split = \
#   [
#    (holdout, (0.5, seed)),
#    (stratified_holdout, (1.0/3.0, seed)),
#    (repeated_holdout, (1.0/3.0, 2, seed)),
#    ...
#   ]
```

Listing 1: Commented split methods

2.2.2 Step b) Apply DecisionTreeClassifier

The `DecisionTreeClassifier` was enabled in the `list_func_classifier` list:

```
list_func_classifier = \
  [
   (DecisionTreeClassifier, ())
  ]
```

Listing 2: Enabled Decision Tree Classifier

At this stage, running the script results in no output because:

1. No train-test split method is active
2. The main loop has no iterations to perform

2.2.3 Step c) Apply Holdout Split Method

The holdout split method was enabled with a 50% train-test split:

```
list_func_tt_split = \
  [
   (holdout, (0.5, seed)),
  ]
```

Listing 3: Enabled holdout split

Additionally, the `accuracy_score` metric was enabled:

```
list_score_metric = \
  [
   (accuracy_score, {}),
  ]
```

Listing 4: Enabled accuracy metric

2.3 Holdout Split Method

The holdout method is the simplest train-test split technique:

- **Definition:** Randomly splits the dataset into two disjoint subsets
- **Parameters:**
  - `test_size=0.5` – 50% of data for testing
  - `seed=5` – Random seed for reproducibility
- **Characteristics:**
  - Single split execution (n_splits=1)
  - Fast and simple
  - High variance in results (depends on random split)

2.4 Results

Using the Pima Indians Diabetes dataset (768 instances, 8 features):

```
train_test_split: holdout
classifier: DecisionTreeClassifier
score_method: accuracy_score
::all-evaluated-datasets::
 66.15% |
66.15% (+/- 0.00%)
```

2.4.1 Analysis

- **Accuracy:** 66.15% – The classifier correctly predicted diabetes status in approximately 2/3 of test cases
- **Standard Deviation:** 0.00% – Expected, as only one split was performed
- **Interpretation:** This represents a baseline performance for the Decision Tree on this medical diagnosis task

3 Exercise 7: Multiple Metrics and Repeated Splits

3.1 Overview

Exercise 7 extends the previous work by implementing the complete scoring pipeline with multiple evaluation metrics and repeated train-test splits. This exercise also addresses important warnings that arise when working with small datasets or imbalanced class distributions.

3.2 Implementation Steps

3.2.1 Step d) Complete the score_recipe Function

The `score_recipe` function required two critical implementations:

```
1  def score_recipe( classifier, X, y, tt_split_indexes,
2                    f_score, **keyword_args_score ):
3      score_all_list = list()
4
5      for ( train_index, test_index ) in tt_split_indexes.split( X,
           y ):
6        X_train, y_train = X[train_index], y[train_index]
7        X_test,  y_test  = X[test_index],  y[test_index]
8
9        # fit (build) model using classifier
10       classifier.fit( X_train, y_train )
11
12       # predict using the model and X_test
13       y_predict = classifier.predict( X_test )
14
15       # score the model using y_test and y_predict
```

```
      score = f_score( y_test, y_predict, **keyword_args_score )

      score_all_list.append( score )

    return score_all_list
```

Listing 5: Completed score_recipe function

Key implementations:

1. `y_predict = classifier.predict( X_test )` – Generates predictions for test data
2. `score = f_score( y_test, y_predict, **keyword_args_score )` – Calculates the metric score

3.2.2 Step e) Test with Repeated Holdout

The configuration was changed to use repeated holdout:

```
list_func_tt_split = \
  [
   (repeated_holdout, (1.0/3.0, 2, seed)),
  ]
```

Listing 6: Repeated holdout configuration

All five evaluation metrics were enabled:

```
list_score_metric = \
  [
   (accuracy_score, {}),
   (precision_score, {"average":"weighted"}),
   (recall_score, {"average":"weighted"}),
   (f1_score, {"average":"weighted"}),
   (cohen_kappa_score, {}),
  ]
```

Listing 7: All evaluation metrics

3.3 Repeated Holdout Method

- **Definition:** Performs multiple random train-test splits
- **Parameters:**
  - `test_size=1.0/3.0` – 33.33% for testing, 66.67% for training
  - `n_splits=2` – Performs 2 independent splits
  - `seed=5` – Random seed for reproducibility
- **Advantages:**
  - Reduces variance compared to single holdout
  - Provides standard deviation estimates
  - More reliable performance estimation

3.4 Evaluation Metrics

3.4.1 Accuracy

- **Definition:** Ratio of correct predictions to total predictions
- **Formula:** Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$
- **Use case:** Overall correctness measure
- **Limitation:** Can be misleading with imbalanced datasets

3.4.2 Precision

- **Definition:** Ratio of true positives to all positive predictions
- **Formula:** Precision = $\frac{TP}{TP+FP}$
- **Use case:** Measures the ability not to label negative samples as positive
- **Question answered:** "Of all instances predicted as positive, how many are actually positive?"

3.4.3 Recall (Sensitivity)

- **Definition:** Ratio of true positives to all actual positive samples
- **Formula:** Recall = $\frac{TP}{TP+FN}$
- **Use case:** Measures the ability to find all positive samples
- **Question answered:** "Of all actual positive instances, how many did we correctly identify?"

3.4.4 F1-Score

- **Definition:** Harmonic mean of precision and recall
- **Formula:** F1 = $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
- **Use case:** Balanced measure when you need to consider both false positives and false negatives
- **Range:** 0 (worst) to 1 (best)

3.4.5 Cohen's Kappa

- **Definition:** Measures agreement between predicted and actual classifications, accounting for chance agreement
- **Formula:** $\kappa = \frac{p_o - p_e}{1 - p_e}$
  - p_o = observed agreement
  - p_e = expected agreement by chance

- **Range:** -1 to +1
  - $\kappa = 1$: Perfect agreement
  - $\kappa = 0$: Agreement by chance
  - $\kappa < 0$: Less than chance agreement

3.5 Results with Simple Dataset

Using the `simple_dataset` (6 instances, 2 features, 2 classes):

```
train_test_split: repeated_holdout
classifier: DecisionTreeClassifier

score_method: accuracy_score
::all-evaluated-datasets::
 50.00% | 0.00% |
25.00% (+/- 25.00%)

score_method: precision_score
::all-evaluated-datasets::
 100.00% | 0.00% |
50.00% (+/- 50.00%)

score_method: recall_score
::all-evaluated-datasets::
 50.00% | 0.00% |
25.00% (+/- 25.00%)

score_method: f1_score
::all-evaluated-datasets::
 66.67% | 0.00% |
33.33% (+/- 33.33%)

score_method: cohen_kappa_score
::all-evaluated-datasets::
 0.00% | 0.00% |
0.00% (+/- 0.00%)
```

3.5.1 Analysis of Results

- **Accuracy:** 25.00% ± 25.00% – High variance due to small dataset
- **Precision:** 50.00% ± 50.00% – One split had 100%, another had 0%
- **Recall:** 25.00% ± 25.00% – Similar variance to accuracy
- **F1-Score:** 33.33% ± 33.33% – Reflects the imbalance in precision/recall
- **Cohen's Kappa:** 0.00% ± 0.00% – No better than random chance

The extremely high standard deviations and poor performance are expected with only 6 samples split into even smaller train/test sets.

3.6 Understanding Metric Warnings

3.6.1 Step f) & g) Warning Analysis

When running the code, two types of warnings appear:

Warning 1: "No Predicted Samples"

```
UndefinedMetricWarning: Precision is ill-defined and being
set to 0.0 in labels with no predicted samples.
```

Explanation:

- Occurs when the classifier **never predicts** a certain class
- Example: Test set contains both class 0 and class 1, but the classifier only predicts class 1
- **Formula breakdown:** $\text{Precision}_{\text{class 0}} = \frac{TP_0}{TP_0+FP_0}$
- If class 0 is never predicted: $TP_0 = 0$ and $FP_0 = 0 \Rightarrow$ division by zero
- Affects: **Precision** and **F1-score**

Warning 2: "No True Samples"

```
UndefinedMetricWarning: Recall is ill-defined and being
set to 0.0 in labels with no true samples.
```

Explanation:

- Occurs when a certain class **doesn't exist** in the test set
- Example: Test set only contains class 1 (no class 0 instances)
- **Formula breakdown:** $\text{Recall}_{\text{class 0}} = \frac{TP_0}{TP_0+FN_0}$
- If no true class 0 samples exist: $TP_0 = 0$ and $FN_0 = 0 \Rightarrow$ division by zero
- Affects: **Recall** and **F1-score**

Why F1-Score Gets Both Warnings The F1-score is the harmonic mean of precision and recall:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Therefore:

- If precision is undefined $\Rightarrow$ F1 is undefined
- If recall is undefined $\Rightarrow$ F1 is undefined
- F1-score inherits warnings from both metrics

3.6.2 Step h) Handling Warnings with zero_division Parameter

Scikit-learn provides the `zero_division` parameter to handle these edge cases:

```
list_score_metric = \
  [
   (accuracy_score, {}),
   (precision_score, {"average":"weighted", "zero_division":0}),
   (recall_score, {"average":"weighted", "zero_division":0}),
   (f1_score, {"average":"weighted", "zero_division":0}),
   (cohen_kappa_score, {}),
  ]
```

Listing 8: Adding zero_division parameter

Effect of zero_division=0:

- Sets undefined metrics to 0.0 instead of raising warnings
- Explicitly handles division by zero cases
- More appropriate than default behavior (which also sets to 0 but warns)
- Warnings are still generated but can be controlled separately

3.6.3 Step i) Suppressing Warnings

To completely suppress the warnings (with caution):

```
warnings.filterwarnings( "ignore", category=
   UndefinedMetricWarning )
```

Listing 9: Suppressing UndefinedMetricWarning

Important considerations:

- **Advantage:** Cleaner output, no warning clutter
- **Disadvantage:** Loss of diagnostic information
- **Best practice:** Only use "ignore" when you fully understand the implications
- **Recommendation:** Keep warnings during development, suppress only in production if necessary

4 Comparative Analysis

4.1 Impact of Dataset Size

Aspect	Simple Dataset (6)	Diabetes Dataset (768)
Train size (50%)	3 instances	384 instances
Test size (50%)	3 instances	384 instances
Accuracy	25% ± 25%	66.15% ± 0%
Warnings	Frequent	Rare
Reliability	Very low	High

Table 1: Impact of dataset size on evaluation

4.2 Holdout vs Repeated Holdout

Characteristic	Holdout	Repeated Holdout
Number of splits	1	Multiple (e.g., 2, 5, 10)
Standard deviation	Always 0%	Provides variance estimate
Reliability	Lower	Higher
Computation time	Faster	Slower
Best for	Quick testing	Reliable evaluation

Table 2: Comparison of split methods

5 Conclusions

5.1 Key Learnings

1. **Train-Test Split Importance:**
   - Proper data splitting is crucial for unbiased evaluation
   - Single holdout provides quick results but high variance
   - Repeated splits offer more reliable performance estimates
2. **Multiple Metrics Matter:**
   - Accuracy alone is insufficient for comprehensive evaluation
   - Precision and recall provide complementary insights
   - F1-score balances both metrics
   - Cohen's Kappa accounts for chance agreement
3. **Edge Cases and Warnings:**
   - Small datasets frequently trigger metric warnings
   - Understanding "no predicted samples" vs "no true samples" is essential
   - The `zero_division` parameter provides explicit control
   - Warning suppression should be used judiciously
4. **Dataset Size Impact:**
   - Minimum viable dataset size is crucial for reliable evaluation
   - With 6 samples, results are essentially random
   - Larger datasets (768+) provide stable, meaningful metrics

5.2 Best Practices

- Use repeated splits (holdout, k-fold) for reliable evaluation
- Report multiple metrics, not just accuracy
- Understand warning messages before suppressing them

- Use `zero_division` parameter explicitly
- Ensure adequate dataset size for your evaluation strategy
- Set random seeds for reproducibility
- Consider stratified splits for imbalanced datasets

5.3 Future Work

Potential extensions to these exercises:

- Implement k-fold cross-validation for more robust evaluation
- Compare performance across multiple classifiers (Naive Bayes, SVM, Random Forest)
- Apply stratified splits to handle class imbalance
- Analyze learning curves with varying dataset sizes
- Implement bootstrap sampling for variance estimation